

Rúbrica de Evaluación - Semana 05

Composición de Componentes y Context API

Competencias a Evaluar

Competencia	Descripción
Composición	Crear componentes flexibles usando children y slots
Compound Components	Implementar patrones de componentes compuestos
Context API	Usar Context para compartir estado global
TypeScript	Tipar correctamente providers y consumers

Distribución de la Evaluación

Evidencia	Peso	Descripción
Proyecto 	100%	Sistema de Temas con Panel de Configuración

 **Nota:** La teoría y los ejercicios son recursos de aprendizaje para prepararte para el proyecto. No son entregables evaluados.

Recursos de Aprendizaje (No Evaluados)

Teoría

Material de estudio para comprender los conceptos:

- Composición vs herencia en React
- Children y ReactNode tipados
- Patrones de compound components
- Context API con TypeScript

Ejercicios Guiados

Práctica para dominar las técnicas antes del proyecto:

Ejercicio	Objetivo de Aprendizaje
01 - Children Básico	Tipar children, ReactNode, ReactElement
02 - Slots Pattern	Named slots, composición flexible
03 - Compound Components	API compuesta, estado compartido interno
04 - Context Básico	createContext, Provider, useContext
05 - Context Avanzado	Reducer, separación, optimización

Criterios de Evaluación del Proyecto (100%)

Distribución de Puntos

Aspecto	Peso	Descripción
Funcionalidad	40%	Features completas y funcionando correctamente
Adaptación al Dominio	35%	Implementación coherente con dominio asignado
Calidad del Código	25%	TypeScript, organización, buenas prácticas

Funcionalidad (40%)

Criterio	Excelente (100%)	Bueno (80%)	Suficiente (70%)	Insuficiente (<70%)
Theme Context	Tema global con persistencia	Tema funcional con toggle	Toggle básico	No implementa tema
Componentes compuestos	API intuitiva y flexible	Componentes bien estructurados	Estructura básica correcta	Componentes mal diseñados
Integración	Todos los componentes integrados	Mayoría integrados	Integración parcial	Sin integración

Adaptación al Dominio (35%)

Criterio	Excelente (100%)	Bueno (80%)	Suficiente (70%)	Insuficiente (<70%)
Coherencia	Perfecta adaptación al dominio asignado	Buena adaptación	Adaptación parcial	No adapta al dominio
Originalidad	Solución creativa y única	Implementación propia	Solución básica	Copia evidente
Contexto	Datos y UI coherentes con el negocio	Mayoría coherente	Parcialmente coherente	Incoherente

Calidad del Código (25%)

Criterio	Excelente (100%)	Bueno (80%)	Suficiente (70%)	Insuficiente (<70%)
TypeScript	Tipos estrictos sin any	Buenos tipos	Tipos básicos	Uso excesivo de any
Código limpio	Excelente organización y documentación	Código bien estructurado	Legible pero mejorable	Desorganizado
Patrones	Aplica patrones correctamente	Usa patrones básicos	Intenta usar patrones	No usa patrones

 Desglose del Proyecto (100%)

Componente	Puntos	Criterios
ThemeProvider	20	Context tipado, toggle funcional
Compound Card	20	Card.Header, Card.Body, Card.Footer
Compound Modal	20	Modal.Trigger, Modal.Content, Modal.Close

Compound Tabs	20	Tabs.List, Tabs.Tab, Tabs.Panel
Integración y Dominio	20	Composición de contextos + dominio único
Total	100	

✓ Lista de Verificación

📖 Preparación (No Evaluada)

- ☐ Leer material teórico de composición y Context API
- ☐ Completar ejercicios guiados para practicar técnicas
- ☐ Revisar ejemplos de compound components
- ☐ Entender flujo Provider → Consumer
- ☐ Practicar tipado de children y ReactNode

🏠 Proyecto (Evaluado - 100%)

- ☐ Sistema de tema claro/oscuro funcional
- ☐ ThemeProvider con Context tipado
- ☐ Al menos 2 compound components implementados
- ☐ Código TypeScript sin errores (sin any)
- ☐ Componentes reutilizables y flexibles
- ☐ Implementación coherente con dominio asignado
- ☐ Documentación clara del uso

🏆 Niveles de Logro

Nivel	Rango	Descripción
Experto	95-100%	Domina composición y Context, código ejemplar
Avanzado	85-94%	Implementa patrones correctamente con buen tipado
Competente	75-84%	Entiende y aplica conceptos principales
En desarrollo	70-74%	Necesita práctica adicional
Insuficiente	<70%	Requiere refuerzo de fundamentos

✓ Criterios de Aprobación

- **Mínimo 70%** en el proyecto para aprobar la semana
- El proyecto debe ser funcional y ejecutable
- Implementación coherente con el dominio asignado
- Código original (sin copias de otros aprendices)

📌 Notas para el Evaluador

1. **Composición:** Verificar que no se use herencia de clases
2. **Tipado de children:** Revisar uso de ReactNode vs tipos específicos
3. **Context:** Evaluar si el valor por defecto es seguro (no undefined sin manejo)
4. **Compound Components:** Validar que el estado se comparte internamente
5. **Dominio único:** Verificar implementación coherente con el dominio asignado

Formato de Entrega

```
week-05-entrega/  
├─ README.md           # Descripción del proyecto y dominio  
├─ src/  
│  ├─ contexts/        # ThemeContext y otros contextos  
│  ├─ components/      # Compound components (Card, Modal, Tabs)  
│  └─ App.tsx          # Integración de componentes  
├─ package.json  
└─ tsconfig.json
```

Entregable único: Proyecto comprimido o enlace a repositorio

Recursos de Apoyo

- [React Patterns](#)
 - [TypeScript + React Cheatsheet](#)
 - [Kent C. Dodds - Compound Components](#)
-